Harava on a Server

Installing Harava next to other sites with nginx and Let's Encrypt, users and permissions, updating, backups and troubleshooting

Otso Laakso (OH2GAX)

Example: `harava.oh2gax.org` on an AWS Ubuntu 24.04 LTS server that also runs WSPR Logger

In short. Harava is a Flask program. On your own computer it opens without a sign-in. On a server it runs as a *systemd* service on `127.0.0.1:5057`, and *nginx* — the same nginx that already serves your other sites — passes `https://harava.oh2gax.org` to it, with a free Let's Encrypt certificate. Every page then needs a sign-in, and every user has one permission: *View only*, *Analyze* or *Admin*.

- Section 1: how the parts fit together, and what the installation adds and does *not* change;
- Section 2: the installation, step by step, with every command;
- Section 3: users, permissions and the Maintenance page;
- Section 4: updating with `update.sh`, the same way as WSPR Logger;
- Section 5: the Workbench, where unknown radios become rules on the server;
- Sections 6–9: backups, security, troubleshooting and a quick reference.

Contents

1 How Harava runs on a server **2**

2 Installing, step by step **2**

2.1 Check first (changes nothing) . . . 2
2.2 Packages . . . 2
2.3 The code and a Python environment . . . 3
2.4 Settings . . . 3
2.5 The map key . . . 3
2.6 The first admin . . . 3
2.7 The service . . . 3
2.8 nginx — this site only . . . 3
2.9 HTTPS with Let's Encrypt . . . 4
2.10 Check the other sites, then sign in . . . 4

3 Users and permissions **4**

4 Updating with update.sh **6**

5 The Workbench **7**

6 Backups, moving and removing **9**

7 Security in short **10**

8 Troubleshooting **10**

9 Quick reference **11**

1 How Harava runs on a server

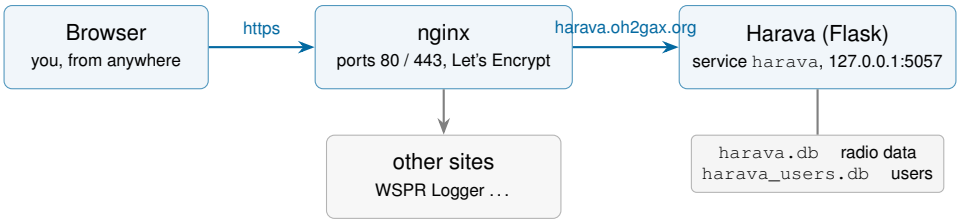


Figure 1. Only nginx is reachable from the internet. It sends the requests for `harava.oh2gax.org` to Harava and the others to their own programs, as before.

What the installation adds. A folder `~/harava` (the program, its settings and databases), the service file `/etc/systemd/system/harava.service`, one nginx site file `/etc/nginx/sites-available/harava.oh2gax.org` with its link in `sites-enabled`, and a certificate for `harava.oh2gax.org`.

What it does not change. The other nginx sites and their certificates, the other services (WSPR Logger runs on its own port, 5008), the firewall and the AWS security group: Harava listens only on `127.0.0.1`, so no new port is opened. Every nginx change is tested with `nginx -t` before nginx is *reloaded* (not restarted); if the test fails, nginx keeps running with the old setup.

Table 1. The files of a server installation. Only the first group comes from GitHub; the rest are local and never in git.

File	What it is
`harava.py, templates/, static/`	the program (from GitHub)
`deploy/harava.service`	the systemd unit to copy (from GitHub)
`deploy/harava.oh2gax.org.nginx`	the nginx site to copy (from GitHub)
`update.sh`	the update script (from GitHub)
`config.ini`	the server's settings (Section 2.4)
`api_keys.py`	the CARTO map key
`harava.db`	the radio data: imports and My devices
`harava_users.db`	users, permission, password hashes and the sign-in log
`.harava_secret`	the random key that signs the sign-in cookies
`data/`	Fieldwatch exports copied to the server (optional; uploads go straight into the database)
`venv/`	the Python environment

2 Installing, step by step

Log in with SSH (the AWS console) as the user `ubuntu`. Replace the domain, the user and the folder if yours differ.

2.1 Check first (changes nothing)

```
dig +short harava.oh2gax.org          # the server's public IP (13.60.119.254)
curl -s https://checkip.amazonaws.com # the same IP
sudo ss -ltnp | grep ':5057 ' || echo "port 5057 is free"
sudo nginx -t                         # nginx must be OK before you start
ls /etc/nginx/sites-enabled/          # your existing sites
python3 --version                     # 3.9 or newer
```

If port 5057 is taken, use another one in `config.ini` (step 2.4) and in the nginx file (step 2.8).

2.2 Packages

```
sudo apt update
sudo apt install -y git python3-venv
```

```
certbot --version || sudo apt install -y certbot python3-certbot-nginx
```

nginx and certbot are already there for the other sites; nothing is upgraded or reconfigured.

2.3 The code and a Python environment

```
cd ~
git clone https://github.com/oh2gax/harava.git
cd harava
python3 -m venv venv
venv/bin/pip install -r requirements.txt
```

2.4 Settings

```
cat > config.ini <<'EOF'
[app]
host = 127.0.0.1
port = 5057
database = harava.db
watch_folder = data
open_browser = false
travel_min_m = 500
require_login = true
behind_proxy = true
session_days = 30
EOF
```

- `host = 127.0.0.1` — only nginx on the same machine reaches Harava.
- `require_login = true` — Harava refuses to start until an admin exists, so it is never on the internet without a sign-in.
- `behind_proxy = true` — Harava trusts nginx's `X-Forwarded-*` headers: the sign-in log shows the real addresses, and the sign-in cookie is sent over HTTPS only.
- `session_days` — how long Keep me signed in lasts.

2.5 The map key

```
cp ~/wspr_logger/api_keys.py ~/harava/api_keys.py   # the same key as WSPR Logger
chmod 600 ~/harava/api_keys.py
```

2.6 The first admin

```
venv/bin/python harava.py --add-admin
```

It asks for a username, first and last name and a password (at least 12 characters, twice). The password is stored only as a hash in `harava_users.db`. All other users are added in the browser (Section 3).

2.7 The service

```
sudo cp deploy/harava.service /etc/systemd/system/harava.service
sudo systemctl daemon-reload
sudo systemctl enable --now harava
systemctl status harava --no-pager
curl -s http://127.0.0.1:5057/healthz        # ok
curl -sI http://127.0.0.1:5057/ | head -3    # 302, Location: /login
```

The unit runs `venv/bin/python harava.py -no-browser` as `ubuntu` in `/home/ubuntu/harava`, restarts it if it stops, and starts it after a reboot. It also sets `TZ=Europe/Helsinki`: the server itself runs in UTC, and without it the days would change at 2 or 3 a.m. Finnish time; change the zone in the unit if yours is different. If your user or folder differs, edit the copy with `sudonano/etc/systemd/system/harava.service` before `daemon-reload`. The log: `journalctl -u harava -f`.

2.8 nginx — this site only

```
sudo cp deploy/harava.oh2gax.org.nginx /etc/nginx/sites-available/harava.oh2gax.org
sudo ln -s /etc/nginx/sites-available/harava.oh2gax.org /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
curl -sI http://harava.oh2gax.org/ | head -3   # 302 to /login
```

The file handles only `server_name harava.oh2gax.org`, accepts uploads up to 512 MB, waits up to 5 minutes for a big import, and passes everything to `127.0.0.1:5057`.

If nginx -t reports an error, nothing has been reloaded and the other sites keep running. Remove the link (`sudorm/etc/nginx/sites-enabled/harava.oh2gax.org`), fix the file and try again.

2.9 HTTPS with Let's Encrypt

```
sudo certbot --nginx -d harava.oh2gax.org
sudo nginx -t
curl -s https://harava.oh2gax.org/healthz    # ok
sudo certbot renew --dry-run                 # renewal works
```

Certbot gets a certificate for this one name and edits only the Harava site file: it adds the HTTPS lines and, when you choose it, the redirect from `http` to `https`. The certbot timer that already renews the other sites' certificates renews this one too.

2.10 Check the other sites, then sign in

```
systemctl status nginx --no-pager | head -5
curl -sI https://<your WSPR Logger address>/ | head -1   # still 200
```

Open `https://harava.oh2gax.org`, sign in as the admin of step 2.6, add the users on Maintenance and upload Fieldwatch exports on Imports.

3 Users and permissions

Table 2. The permissions. Buttons that need a permission the user does not have are not shown, and the server refuses the request anyway.

Permission	Can
View only	see the map, the Devices list, the Phone check, the Identification page and the guides; export CSV
Analyze	also change the data: upload and import Fieldwatch exports, Scan folder, switch or delete imports, edit My devices, Reload database
Admin	also manage users on the Maintenance page, see and combine every user's data (Whose data → *All users*), and use the Workbench

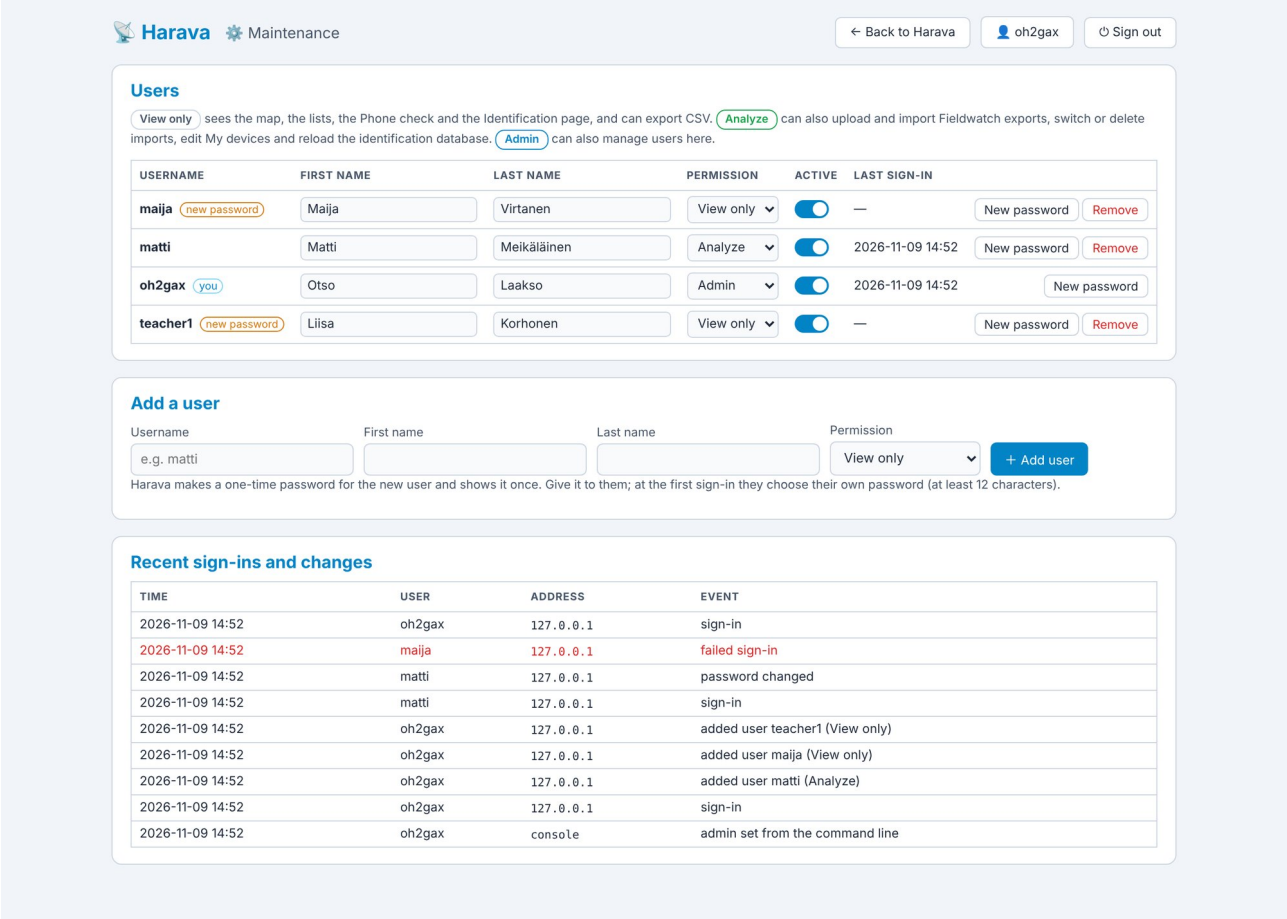


Figure 2. The Maintenance page (`/maintenance`, Maintenance in the menu for admins). Top: the users, with their names, permission and *active* switch, which take effect as soon as they are changed. Middle: adding a user. Bottom: the log of sign-ins, failed sign-ins and changes, with the address they came from.

Each user's own data. Every import belongs to the user who imported it, recorded on one of that user's *phones*. Others do not see it unless the owner shares it (with all users or with chosen users); admins see every import. Each user's *My devices* are personal. Data imported while Harava had no users (on your own computer) belongs to the first admin; when the first user is added, it moves to that admin. A removed user's imports stay, shown as *removed user*, and an admin can delete them.

Adding a user. Type a username (3–64 letters, digits or `.` `_` `@` `-`), first and last name, choose the permission and press Add user. Harava makes a *one-time password* and shows it once (Figure 3); give it to the user. At the first sign-in the user must choose their own password before anything else.

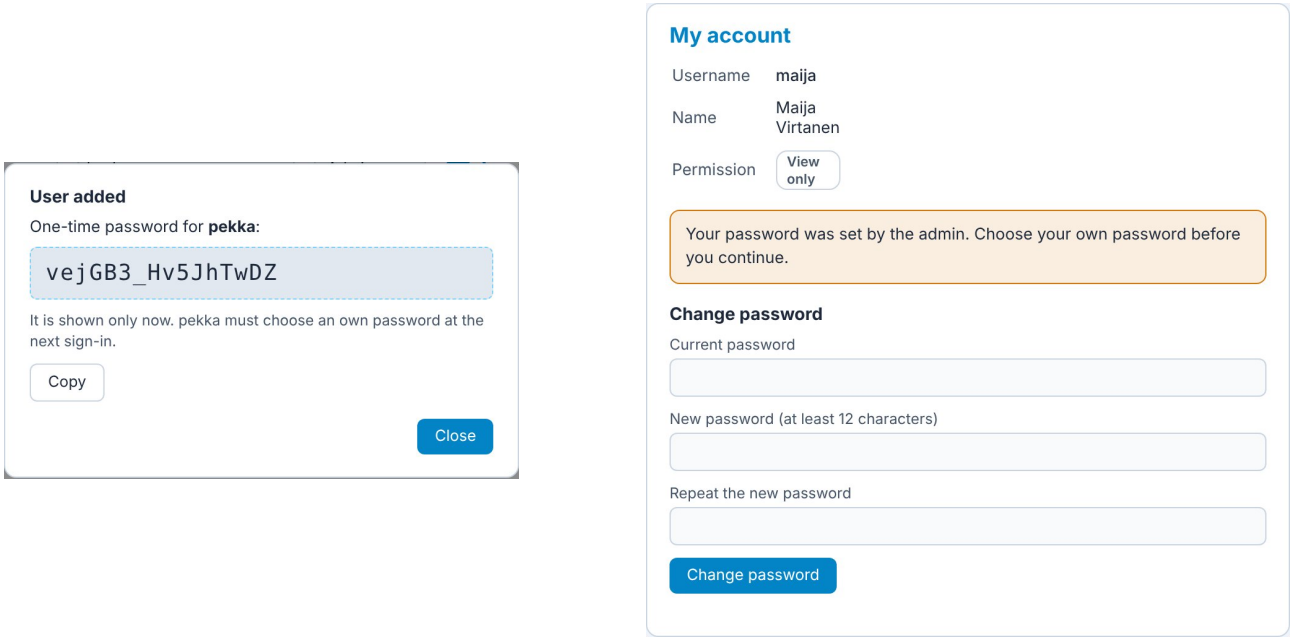


Figure 3. Left: the one-time password of a new user, shown once. Right: *My account* at the user's first sign-in — the own password first.

Changing a user. Edit the names, the permission or the *active* switch in the table. A disabled user cannot sign in, and a signed-in user is signed out at the next click. New password makes a new one-time password (the old one stops working at once); Remove deletes the user — their imports and My devices stay.

Rules that keep you in. There is always at least one active admin: the last admin cannot be removed, disabled or made View only or Analyze. Nobody can remove or disable themselves.

My account. Every user finds *My account* by clicking their name in the menu, and changes their password there. A new password signs out the user's other sessions.

Forgotten admin password. On the server: `cd ~/harava && venv/bin/python harava.py --add-admin` with the same username sets a new password (and makes the user an admin). Then sign in again.

Failed sign-ins. Each failed sign-in waits one second. Five from the same address within 15 minutes lock that address for the rest of the 15 minutes, and 30 from all addresses lock everyone; `sudo systemctl restart harava` clears the counters. Every failed sign-in is in the log on the Maintenance page.

4 Updating with update.sh

New features come from GitHub. On the server, in the SSH console:

```
bash ~/harava/update.sh
```

It shows the new commits and the changed files and asks `Update to <hash>? [y/N]`. After `y` it

1. copies `harava.db`, `harava_users.db` and `ids/local_rules.json` to `~/harava_backups/pre_update_<time>` (safe while Harava runs; the 7 newest copies are kept);
2. checks every changed Python file, page template and JSON file straight from git *before* anything is changed — if one fails, nothing changes and the old version keeps running;
3. updates the files (`git merge --ff-only`) and installs packages only if `requirements.txt` changed;
4. restarts *only* the `harava` service and waits until `/healthz` answers.

Table 3. The options of update.sh.

Option	What it does
`bash update.sh`	show, ask, update, restart, check
`bash update.sh -y`	the same without asking
`bash update.sh -rollback`	go back to the version that ran before the last update, restart and check
`SERVICE=name bash update.sh`	for a service with another name

A new database format. The version of 10 November 2026 stores every hear once per phone and links the imports to them. The first start after `update.sh` converts `harava.db` by itself, in one transaction (all or nothing; about a second for a few days of data), and the existing imports, phone and My devices become the first admin's. `bash update.sh -rollback` to an older version notices the newer format and puts back the database copy made before the update; the newer database is kept next to it as `harava.db.newer-<time>`.

Your local files are safe. `config.ini`, `api_keys.py`, the two databases, `.harava_secret` and `data/` are not in git, and the script never runs `git reset`, `stash` or `checkout`. If GitHub changes a file that has also been edited on the server, it stops without changing anything and tells what to do. It also warns when `api_keys.py` is missing, when `require_login` is not `true`, or when there are no users. Run it as `bash update.sh`; do not `chmod +x` it (git would see the changed mode as a local edit).

5 The Workbench

What it is. Workbench in the menu (`/workbench`, admins only; also a view in the phone app at `/app`) lists every Bluetooth radio of all active imports that Harava cannot name yet. Radios are put in buckets by *what they advertise*, without their address and the bytes that change: the company and the first byte of the manufacturer data, service data, a vendor's own 128-bit UUID, other services and AD types — and the shape of the name only when nothing else tells. One row is one kind of device. Three tabs: *Unknown* (advertising data, no rule), *Weakly identified* (company only, custom UUID only, or a low-confidence rule) and *No data* (flags only).

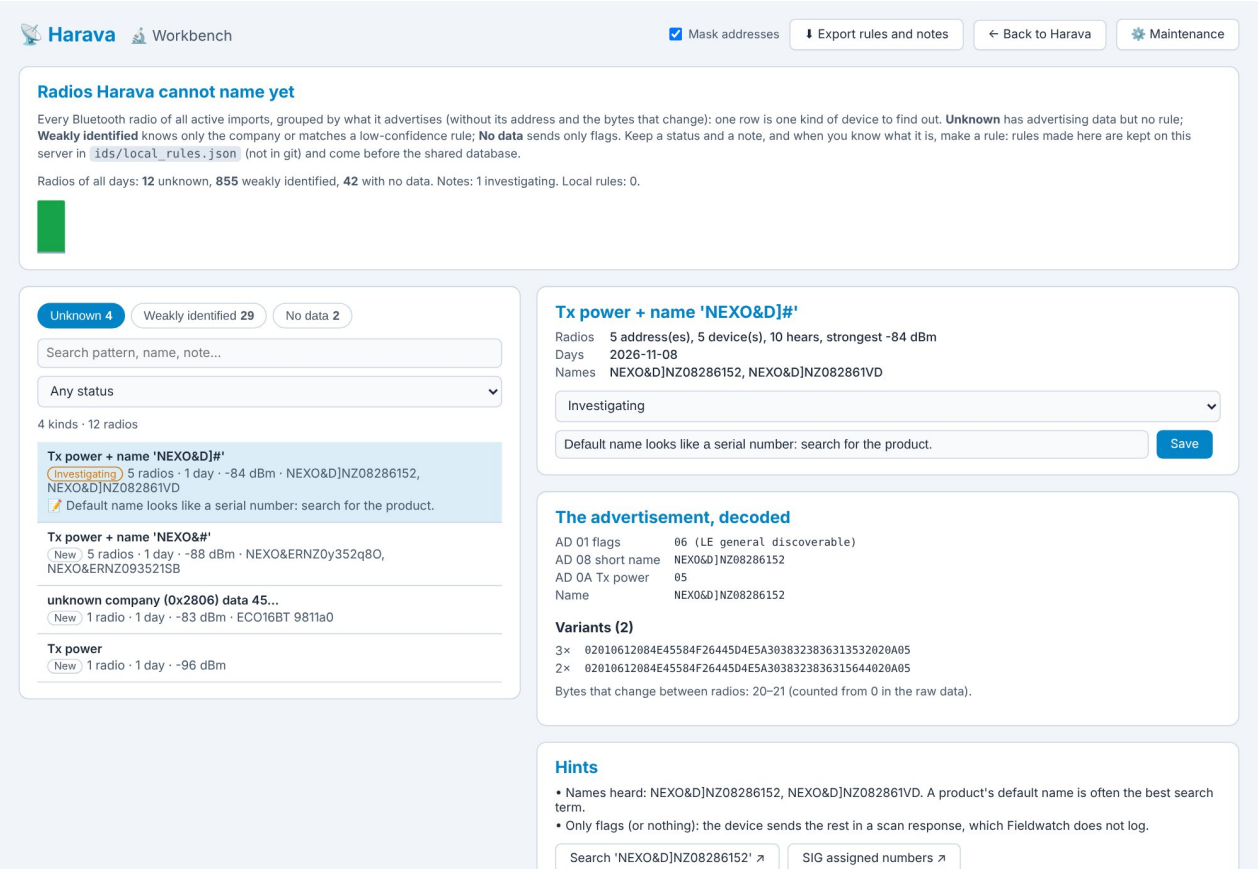


Figure 4. The Workbench: progress per day at the top, the kinds on the left, one kind on the right with its status and note, the advertisement decoded, the variants and the bytes that change, hints and search links.

Notes. Every kind can have a status — New, Investigating, Identified, Ignore, Can't identify — and a note. They are stored in the table `wb_notes` in `harava.db` and are shared by all admins; the fingerprint stays the same when rules change, so a note stays with its kind.

Rules. Make a rule is filled with a match that all the radios of the kind share. Preview counts what it would catch on all days, and which already identified radios it would take over; Save rule checks it (hex fields, UUID lists, a regular expression that compiles, a known layer, at least one condition), gives it an id starting with `local-`, and starts using it at once — no restart.

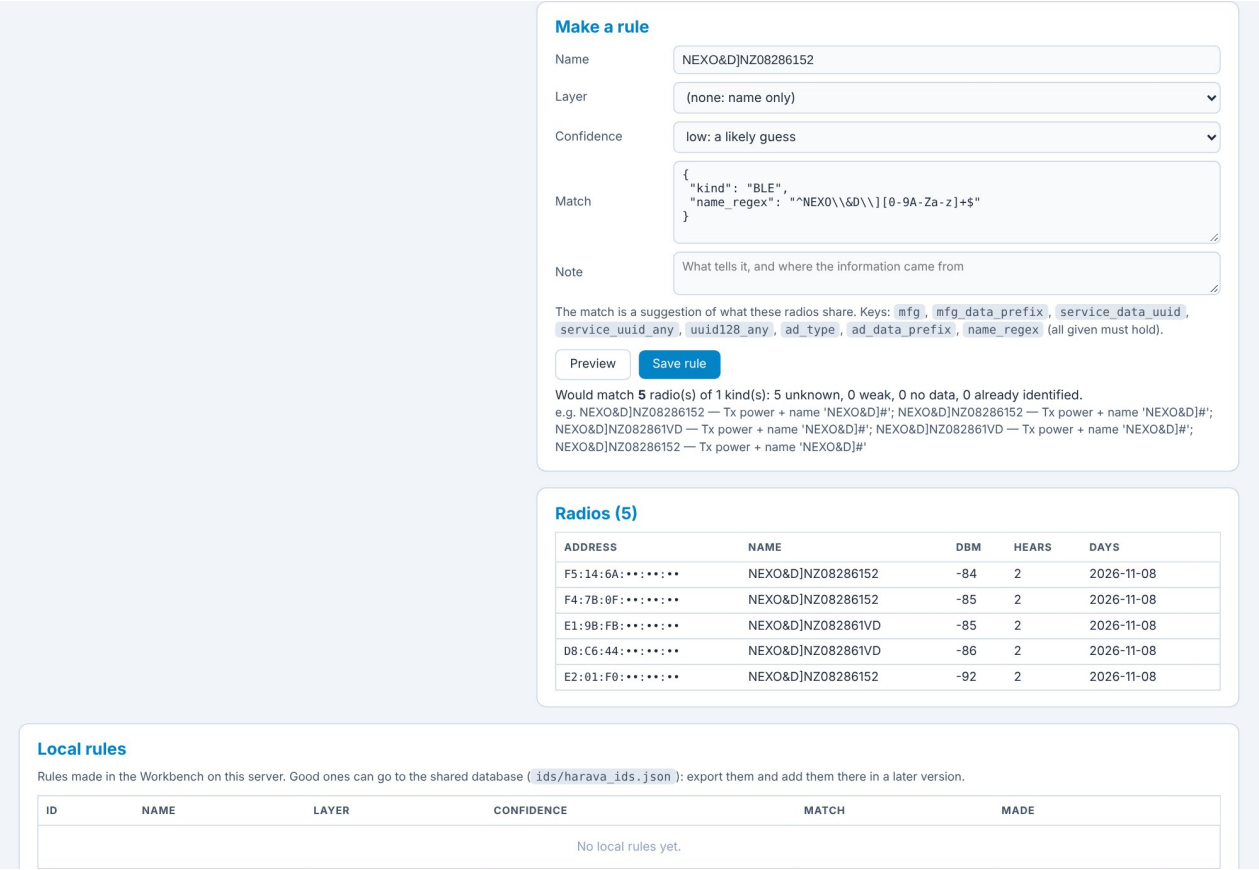


Figure 5. Making a rule: name, layer, confidence and the suggested match, with the preview of what it would catch.

Where local rules live. In `~/harava/ids/local_rules.json`, on this server only. The file is in `.gitignore`, so `update.sh` never touches it, and it copies it with the databases before every update. Local rules are tried *before* the shared `ids/harava_ids.json`. The Local rules table at the bottom of the Workbench lists them and removes them; Export rules and notes downloads the rules and the notes as one JSON file, so good rules can be added to the shared database in a later version of Harava.

6 Backups, moving and removing

What to keep. `harava.db` (also the Workbench notes), `harava_users.db`, `ids/local_rules.json` (the Workbench's rules), `config.ini`, `api_keys.py` and `.harava_secret`. `archive/` holds a compressed copy of every uploaded export (for Re-read; `keep_originals = false` in `config.ini` turns it off); it can be rebuilt from your own exports, so it is not part of the update backups. `update.sh` copies the databases and the local rules before every update; copy them off the server now and then, for example from your own computer:

```
scp ubuntu@harava.oh2gax.org:harava_backups/pre_update_*/harava*.db .
```

A consistent copy while Harava runs: `sqlite3 harava.db ".backup harava_copy.db"` (or the copies `update.sh` makes).

Moving to another server. Install as in Section 2, but before step 2.6 copy the six files above (and `archive/`, if you want Re-read) into `~/harava`; the users and the data come with them.

Removing Harava completely.

```
sudo systemctl disable --now harava
sudo rm /etc/systemd/system/harava.service
sudo rm /etc/nginx/sites-enabled/harava.oh2gax.org
    /etc/nginx/sites-available/harava.oh2gax.org
sudo nginx -t && sudo systemctl reload nginx
sudo certbot delete --cert-name harava.oh2gax.org
rm -rf ~/harava ~/harava_backups
```

7 Security in short

- Passwords are stored only as salted hashes (Werkzeug, scrypt); a one-time password must be changed at the first sign-in; at least 12 characters.
- The sign-in is a signed cookie (`HttpOnly`, `SameSite=Lax`, HTTPS only behind nginx); the key is random and stays on the server in `.harava_secret`. Without Keep me signed in it ends when the browser closes.
- A new password, a disabled user, a removed user or a changed permission takes effect at once, also for a user who is signed in.
- Requests that change data must come from Harava's own page; pages are sent with `no-store`, `X-Frame-Options: DENY` and `nosniff`; search engines are told not to index the site.
- Failed sign-ins are slowed down and locked out (Section 3) and logged with their address.
- The service runs as `ubuntu`, not as root, with `NoNewPrivileges`, a private `/tmp` and `UMask=0077`; `harava_users.db`, `.harava_secret` and `api_keys.py` are readable only by that user.
- Harava stores the addresses and names of other people's radios. Keep the users few, use Mask addresses for screenshots you share, and follow the data-protection law of your country.

8 Troubleshooting

Table 4. Common problems.

Symptom	Cause and fix
502 Bad Gateway	Harava is not running: `systemctl status harava`, `journalctl -u harava -n 50`.
The service stops at once; the log says *require_login = true but there are no users yet*	Add the first admin (step 2.6), then `sudo systemctl restart harava`.
413 Request Entity Too Large when uploading	The file is over 512 MB: raise `client_max_body_size` in the site file (and `MAX_CONTENT_LENGTH` in `harava.py`).
Too many failed sign-ins	Wait 15 minutes, or `sudo systemctl restart harava`.
Forgotten admin password	`venv/bin/python harava.py -add-admin` with the same username.
The sign-in log shows `127.0.0.1` for everyone	`behind_proxy = true` is missing from `config.ini`.
Signed out after every click	The site is opened over `http` while `behind_proxy = true`: open `https://`, or let certbot add the redirect.
The map has an "API key required" watermark	`api_keys.py` is missing or has no key.
`nginx -t` warns *protocol options redefined for 0.0.0.0:443* (or `[::]:443`) after certbot	Another site writes its `listen 443` line differently, e.g. with `http2`. Make Harava's lines match the other sites' (for example `listen [::]:443 ssl http2;` and `listen 443 ssl;`), then `sudo nginx -t && sudo systemctl reload nginx`. The warning itself is harmless.
Days or times are 2–3 hours off	The server runs in UTC: the unit needs `Environment=TZ=Europe/Helsinki`. Copy the unit again, `sudo systemctl daemon-reload`, `sudo systemctl restart harava`.
`update.sh`: *GitHub changes file(s) that are also modified on this machine*	A file from git was edited on the server: copy it aside, `git checkout -- <file>`, run `update.sh` again.

9 Quick reference

I want to …	Command
update Harava	`bash ~/harava/update.sh`
go back after an update	`bash ~/harava/update.sh -rollback`
see if it runs	`systemctl status harava`
read its log	`journalctl -u harava -f`
restart it	`sudo systemctl restart harava`
add an admin / reset a password	`cd ~/harava && venv/bin/python harava.py -add-admin`
test nginx and reload it	`sudo nginx -t && sudo systemctl reload nginx`
test the renewal	`sudo certbot renew -dry-run`
check that Harava answers	`curl -s https://harava.oh2gax.org/healthz`

Table 5. Commands on the server.

Harava is MIT licensed: `https://github.com/oh2gax/harava`. This guide was written with the assistance of Claude (Anthropic).